

RD PARSER FOR DECLARATION STATEMENTS

Name: Satyam Singh

Reg no: 230905256

Roll No: 37

CSE B2

Lab exercise:

Q1) Design the recursive descent parser to parse array declarations and expression statements with error reporting.

Subset of grammar 7.1 is as follows:

```
Program → main () { declarations statement-list }
identifier-list → id | id, identifier-list | id[number], identifier-list | id[number]
statement_list → statement statement_list | ∈
statement → assign-stat;
assign_stat → id = expn
expn → simple-expn eprime
epime → relop simple-expn | ∈
simple-exp → term seprime
seprime → addop term seprime | ∈
term → factor tprime
tprime → mulop factor tprime | ∈
factor → id | num
relop → = | != | <= | >= | > | <
addop → + | -
mulop → * | / | %

statement → assign-stat; | decision_stat | looping-stat
decision-stat → if (expn) {statement_list} dprime
```

Ans)

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

char token[50];

FILE *fp;

void Program();

void identifier_list();

void statement_list();

void statement();

void assign_stat();

void expn();

void eprime();

void simple_expn();

void seprime();

void term();

void tprime();

void factor();

void relop();

void addop();

void mulop();

void match(char *expected);

void error(char *msg);

int getToken();

int getToken()

{

return fscanf(fp,"%s",token);
```

```
}

void error(char *msg)
{
    printf("Syntax Error: %s near token '%s'\n",msg,token);
    exit(1);
}

void match(char *expected)
{
    if(strcmp(token,expected)==0)
    {
        if(getToken()<1)
            strcpy(token,"EOF");
    }
    else
    {
        printf("Syntax Error: expected '%s' but found '%s'\n",expected,token);
        exit(1);
    }
}

void Program()
{
    match("main");
    match("(");
    match(")");
    match("{");

    identifier_list();
    statement_list();
}
```

```
match("}");
}
void identifier_list()
{
match("id");

if(strcmp(token,"[")==0)
{
match("[");
match("num");
match("]");
}

if(strcmp(token,",")==0)
{
match(",");
identifier_list();
}
}
void statement_list()
{
while(strcmp(token,"id")!=0)
{
statement();
}
}
void statement()
```

```
{
    assign_stat();
    match(";");
}

void assign_stat()
{
    match("id");
    match("=");
    expn();
}

void expn()
{
    simple_expn();
    eprime();
}

void eprime()
{
    if(strcmp(token,"==")==0 || strcmp(token,"!=")==0 ||
    strcmp(token,"<")==0 || strcmp(token,">")==0 ||
    strcmp(token,"<=")==0 || strcmp(token,">=")==0)
    {
        relop();
        simple_expn();
    }
}

void simple_expn()
{
    term();
```

```
seprime();
}
void seprime()
{
if(strcmp(token,"+")==0 || strcmp(token,"-")==0)
{
addop();
term();
seprime();
}
}
void term()
{
factor();
tprime();
}
void tprime()
{
if(strcmp(token,"*")==0 || strcmp(token,"/")==0 || strcmp(token,"%")==0)
{
mulop();
factor();
tprime();
}
}
void factor()
{
if(strcmp(token,"id")==0)
```

```
match("id");
```

```
else if(strcmp(token,"num")==0)
```

```
match("num");
```

```
else
```

```
error("Invalid factor");
```

```
}
```

```
void relop()
```

```
{
```

```
if(strcmp(token,"")==0) match("==");
```

```
else if(strcmp(token,"!=")==0) match("!=");
```

```
else if(strcmp(token,"<")==0) match("<");
```

```
else if(strcmp(token,">")==0) match(">");
```

```
else if(strcmp(token,"<=")==0) match("<=");
```

```
else if(strcmp(token,">=")==0) match(">=");
```

```
else error("Invalid relational operator");
```

```
}
```

```
void addop()
```

```
{
```

```
if(strcmp(token,"+")==0) match("+");
```

```
else if(strcmp(token,"-")==0) match("-");
```

```
else error("Invalid addition operator");
```

```
}
```

```
void mulop()
```

```
{
```

```
if(strcmp(token,"*")==0) match("*");
```

```
else if(strcmp(token,"/")==0) match("/");
```

```
else if(strcmp(token,"%")==0) match("%");
else error("Invalid multiplication operator");
}
int main()
{
fp = fopen("input2.txt","r");
if(fp==NULL)
{
printf("Input file not found\n");
exit(1);
}
if(getToken()<1)
{
printf("Empty input file\n");
exit(1);
}
Program();
if(strcmp(token,"EOF")!=0)
error("Unexpected tokens after program");
fclose(fp);
printf("Parsing Successful\n");
return 0;
}
```


Output with error in input:

```
week8 > ≡ input2.txt
1  id = id + ;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ... bash - week8 + v □

```
● cd1-6cse-b2@sce-cl11-03:~/Documents/230905256/week8$ gcc q1.c -o q3
⊙ cd1-6cse-b2@sce-cl11-03:~/Documents/230905256/week8$ ./q3
Syntax Error: expected 'main' but found 'id'
```

Output without Error:

```
week8 > ≡ input.txt
1  main ( ) {
2  id , id [ num ]
3  id = id + num ;
4  id = id * num ;
5  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ... bash - week8 + v □

```
● cd1-6cse-b2@sce-cl11-03:~/Documents/230905256/week8$ gcc q1.c -o qq
● cd1-6cse-b2@sce-cl11-03:~/Documents/230905256/week8$ ./qq
Parsing Successful
```